

INGENIERÍA INFORMÁTICA

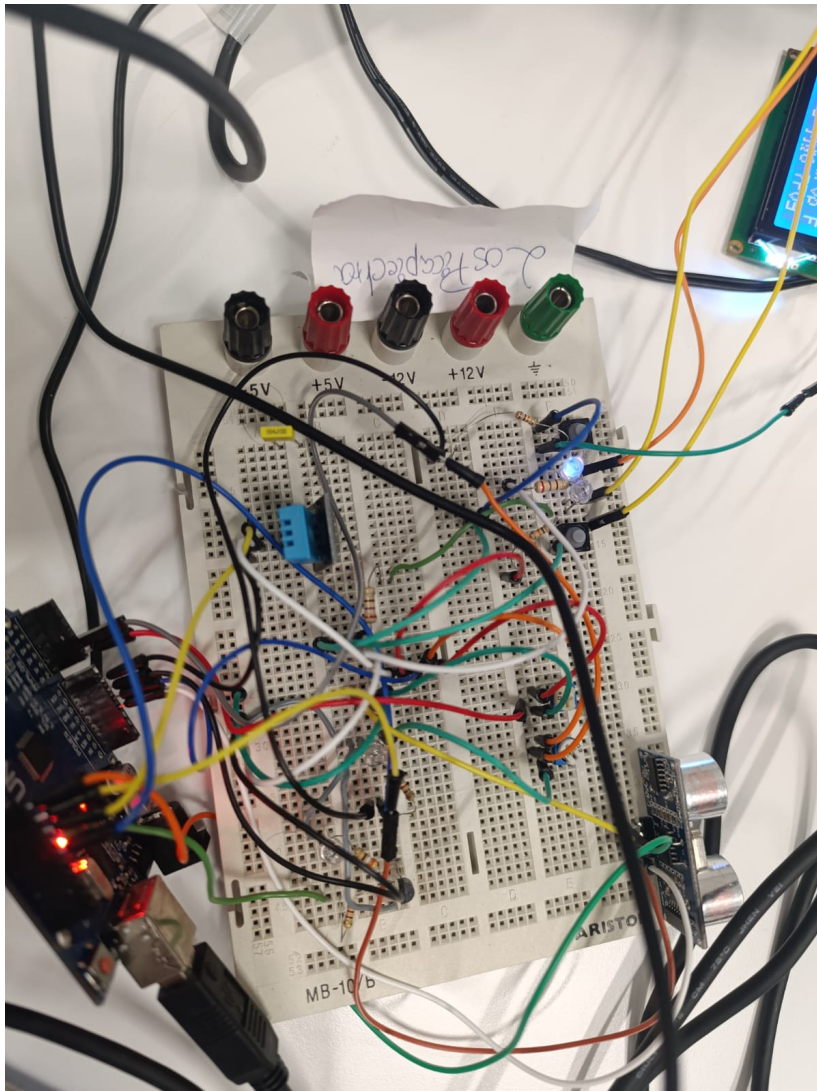
Instalación y Mantenimiento de Computadores

Sistemas Empotrados II

---

**Sistema IoT para control  
y monitorización de un terrario**

---



Montaje final del sistema

**Autor:** Pablo Galilea Valverde  
**Asignatura:** Instalación y Mantenimiento de Computadores  
**Titulación:** Grado en Ingeniería Informática  
**Fecha:** 19 de mayo de 2026  
**Entrega:** 15 de mayo de 2026

# Índice

---

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                               | <b>2</b>  |
| 1.1. Objetivos . . . . .                                             | 2         |
| <b>2. Descripción general del sistema</b>                            | <b>3</b>  |
| 2.1. Arquitectura del sistema . . . . .                              | 3         |
| 2.2. Diagrama de flujo general . . . . .                             | 3         |
| 2.3. Sensores y actuadores . . . . .                                 | 4         |
| 2.4. Protocolo de comunicación serie . . . . .                       | 4         |
| <b>3. Subsistema Arduino UNO</b>                                     | <b>5</b>  |
| 3.1. Descripción funcional . . . . .                                 | 5         |
| 3.2. Esquema eléctrico . . . . .                                     | 5         |
| 3.3. Descripción del circuito de potencia (MOSFET IRL540n) . . . . . | 6         |
| 3.4. Interrupción del Timer1 y lectura de sensores . . . . .         | 6         |
| 3.4.1. Configuración del Timer1 . . . . .                            | 6         |
| 3.4.2. Rutina de servicio de interrupción (ISR) . . . . .            | 7         |
| 3.5. Lógica de control y protocolo de prioridad . . . . .            | 8         |
| <b>4. Subsistema Raspberry Pi 3B+</b>                                | <b>9</b>  |
| 4.1. Descripción funcional . . . . .                                 | 9         |
| 4.2. Fotografías del montaje . . . . .                               | 9         |
| 4.3. Modelo de hilos ( <i>threading</i> ) . . . . .                  | 10        |
| 4.3.1. Anti-rebote por software en el hilo de botones . . . . .      | 11        |
| 4.4. Buffer de datos y promediado para ThingSpeak . . . . .          | 11        |
| 4.5. Detección automática de alarma . . . . .                        | 11        |
| <b>5. Interfaz en la nube: ThingSpeak</b>                            | <b>13</b> |
| 5.1. Configuración del canal . . . . .                               | 13        |
| 5.2. Comunicación REST con ThingSpeak . . . . .                      | 13        |
| 5.3. Resultados obtenidos en ThingSpeak . . . . .                    | 14        |
| <b>6. Presupuesto estimado</b>                                       | <b>16</b> |
| <b>7. Conclusiones</b>                                               | <b>17</b> |
| 7.1. Conclusiones técnicas . . . . .                                 | 17        |
| 7.2. Dificultades encontradas . . . . .                              | 17        |
| <b>8. Referencias</b>                                                | <b>18</b> |
| <b>A. Código fuente Arduino (terrario.ino)</b>                       | <b>19</b> |
| <b>B. Código fuente Raspberry Pi (cucu.py)</b>                       | <b>22</b> |

## 1. Introducción

---

El presente documento recoge la memoria técnica del proyecto final de la asignatura *Instalación y Mantenimiento de Computadores* (Sistemas Empotrados II), consistente en el diseño, implementación y documentación de un sistema IoT para la **automatización y monitorización de un terrario** de pequeñas dimensiones.

El sistema permite vigilar y controlar las condiciones ambientales del terrario (temperatura, humedad y detección de fugas) garantizando el confort de los seres vivos que lo habitan. Para ello se integran dos plataformas hardware —un microcontrolador **Arduino UNO** y un minicomputador **Raspberry Pi 3B+**— junto con la plataforma de visualización en la nube **ThingSpeak**.

### 1.1. Objetivos

- Diseñar e implementar un sistema de adquisición de datos con Arduino UNO que lea temperatura, humedad y distancia por ultrasonidos cada segundo mediante interrupción de *timer* hardware.
- Controlar un ventilador y un sistema de iluminación mediante señal PWM, ajustables tanto manualmente (potenciómetros) como remotamente (ThingSpeak).
- Integrar una interfaz de usuario local con Raspberry Pi 3B+: pantalla LCD, botones de inicio/alarma e indicadores LED.
- Publicar y visualizar los datos del sistema en ThingSpeak con actualización cada 30 segundos.
- Detectar posibles fugas del animal mediante sensor de barrera de ultrasonidos en la puerta del terrario.

## 2. Descripción general del sistema

## 2.1. Arquitectura del sistema

El sistema se compone de tres subsistemas que se comunican formando una arquitectura en dos niveles:

1. **Subsistema Arduino UNO:** nodo sensor/actuador. Adquiere datos de los sensores y controla los actuadores físicos del terrario.
2. **Subsistema Raspberry Pi 3B+:** nodo de control central. Gestiona la comunicación serie con el Arduino, la interfaz de usuario local y la comunicación con la nube.
3. **Plataforma ThingSpeak:** almacena y visualiza los datos, y permite el control remoto de actuadores mediante una interfaz web.

## 2.2. Diagrama de flujo general

La figura 1 muestra el diagrama de flujo del sistema completo, incluyendo la interacción entre los tres subsistemas.

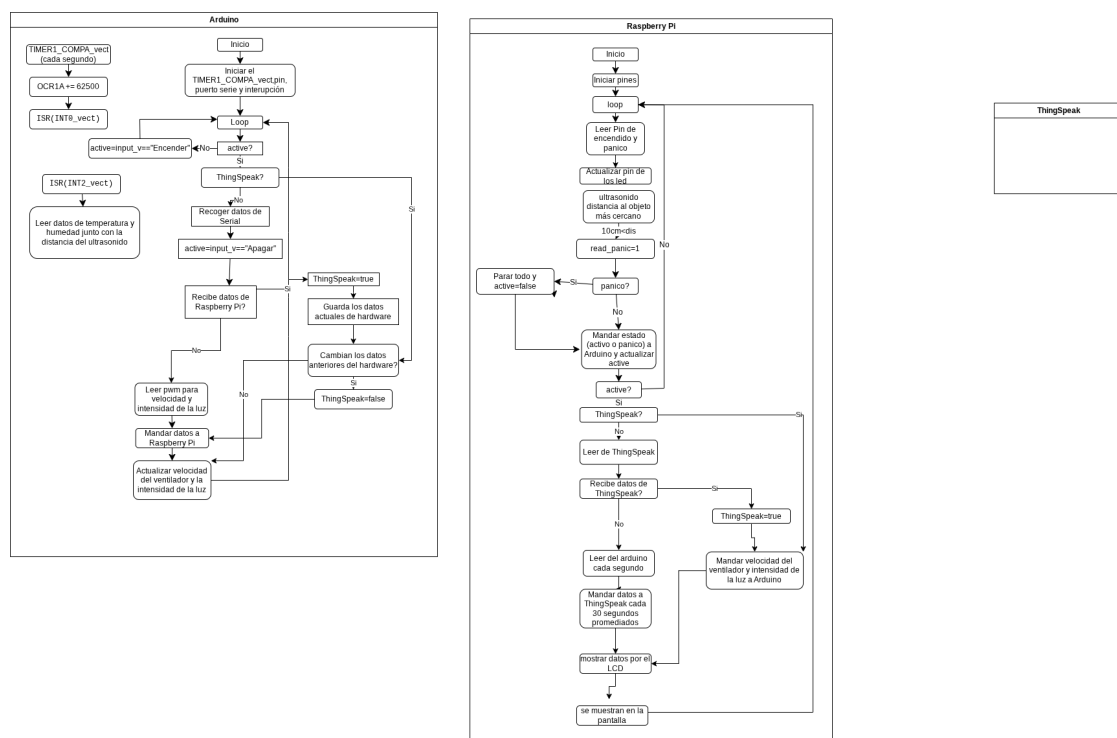


Figura 1: Diagrama de flujo del sistema completo

### 2.3. Sensores y actuadores

Cuadro 1: Sensores y actuadores del sistema

| Componente               | Tipo      | Nodo    | Función                                        |
|--------------------------|-----------|---------|------------------------------------------------|
| <b>DHT11</b>             | Sensor    | Arduino | Temperatura y humedad relativa                 |
| <b>HC-SR04</b>           | Sensor    | Arduino | Distancia por ultrasonidos (detección de fuga) |
| <b>Potenciómetro 1</b>   | Entrada   | Arduino | Control manual de velocidad del ventilador     |
| <b>Potenciómetro 2</b>   | Entrada   | Arduino | Control manual de intensidad de iluminación    |
| <b>Ventilador (PWM)</b>  | Actuador  | Arduino | Regulación de temperatura                      |
| <b>Tira LED (PWM)</b>    | Actuador  | Arduino | Iluminación del terrario                       |
| <b>Botón INICIO</b>      | Entrada   | RPi 3B+ | Arranque/parada del sistema                    |
| <b>Botón ALARMA</b>      | Entrada   | RPi 3B+ | Activación/desactivación manual de alarma      |
| <b>LED ESTADO</b>        | Indicador | RPi 3B+ | Sistema activo/inactivo                        |
| <b>LED ALARMA</b>        | Indicador | RPi 3B+ | Estado de alarma                               |
| <b>Pantalla LCD 16×2</b> | Salida    | RPi 3B+ | Visualización local de datos y estado          |

### 2.4. Protocolo de comunicación serie

La comunicación entre Arduino y Raspberry Pi se realiza por **UART a 9600 bps** sobre USB (/dev/ttyUSB0). El Arduino envía tramas con el siguiente formato:

**V:val L:val T:val H:val D:val\n**

donde V es velocidad del ventilador (0–1023), L intensidad de iluminación (0–1023), T temperatura (°C), H humedad (%) y D distancia (cm). La Raspberry Pi envía comandos de control: **encender**, **apagar**, o los valores enteros de velocidad e intensidad.

### 3. Subsistema Arduino UNO

#### 3.1. Descripción funcional

El Arduino UNO actúa como nodo sensor/actuador. Sus funciones son:

- Leer el DHT11 y el HC-SR04 cada segundo mediante interrupción de *timer* hardware (Timer1).
- Leer los dos potenciómetros analógicos para el control manual de los actuadores.
- Generar señales PWM para el ventilador y la iluminación a través de transistores MOSFET IRL540n.
- Enviar los datos adquiridos a la Raspberry Pi por puerto serie y recibir los valores de control remoto.
- Gestionar la lógica de activación/desactivación del sistema.

#### 3.2. Esquema eléctrico

Las figuras 2 y 3 muestran el esquema eléctrico del subsistema Arduino, tanto en representación visual (Fritzing) como en notación técnica de circuitos.

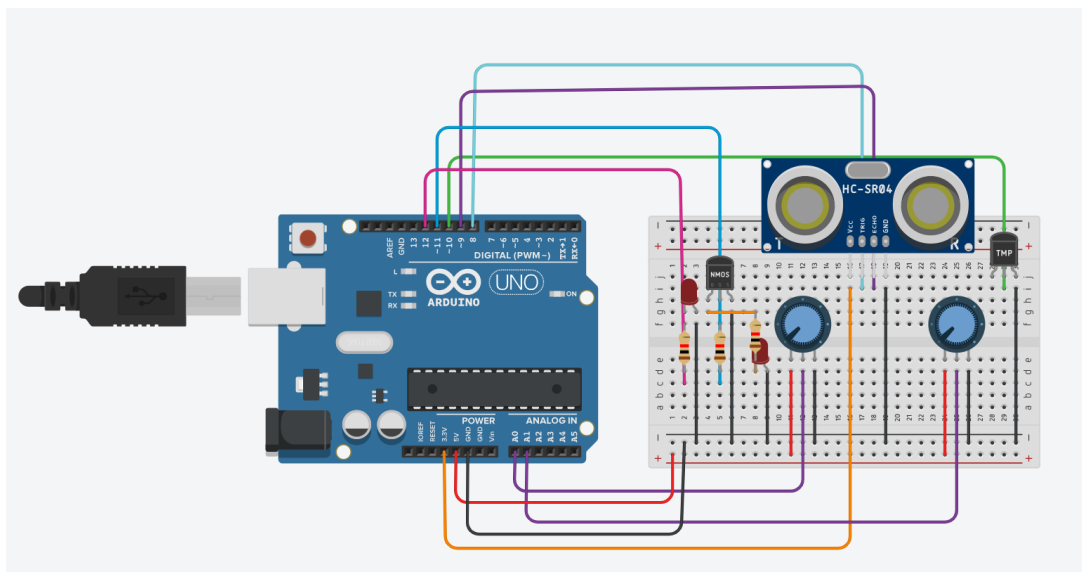


Figura 2: Esquema eléctrico visual del subsistema Arduino (Fritzing)

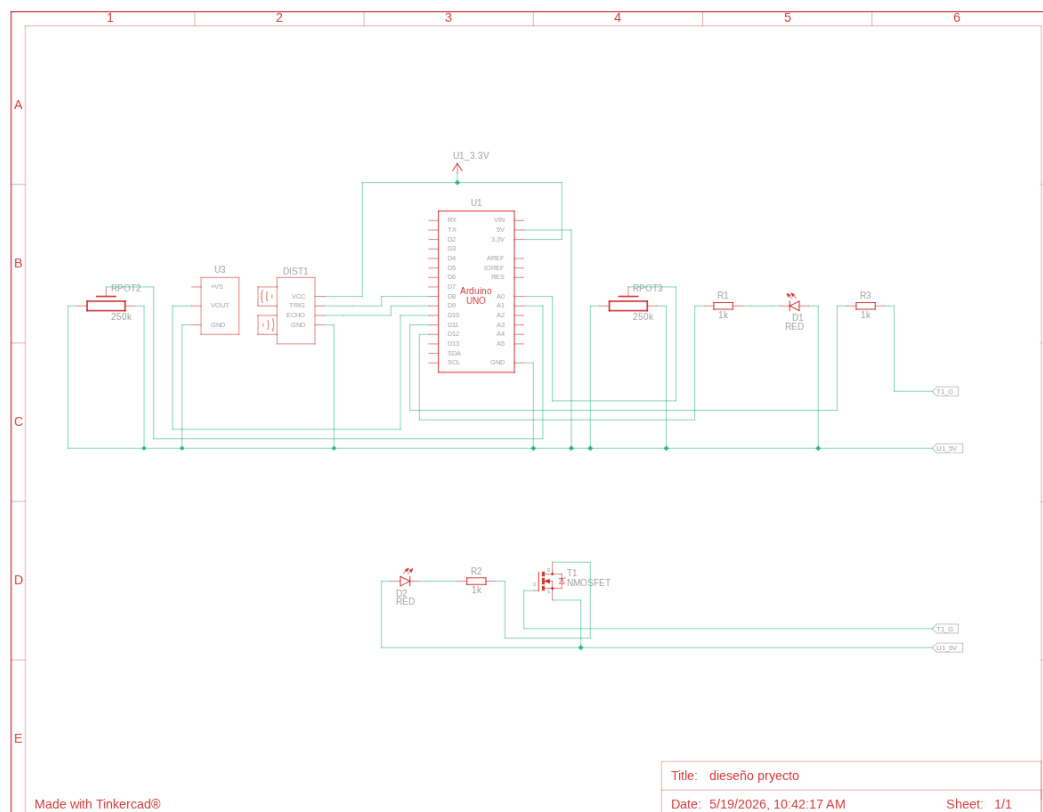


Figura 3: Esquema eléctrico técnico del subsistema Arduino

### 3.3. Descripción del circuito de potencia (MOSFET IRL540n)

El control de los actuadores se realiza mediante transistores MOSFET de canal N **IRL540n**. Este transistor se elige por su baja tensión umbral de puerta ( $V_{GS(th)} \approx 1,0\text{ V}$ ), compatible con los niveles lógicos de 5 V del Arduino, y su baja resistencia de conducción ( $R_{DS(on)} < 77\text{ m}\Omega$ ), que minimiza la disipación de calor.

#### Nota técnica

El circuito de cada actuador incluye una **resistencia de puerta** R1 ( $\approx 220\ \Omega$ ) para limitar la corriente transitoria al cargar la capacidad parásita de la puerta, y una **resistencia** R2 ( $\approx 10\text{ k}\Omega$ ) para garantizar que el MOSFET permanece en corte cuando el pin Arduino está en alta impedancia (por ejemplo, durante el arranque).

La señal PWM del Arduino (pines 11 y 12, frecuencia  $\approx 490\text{ Hz}$ ) varía el ciclo de trabajo del MOSFET, regulando la potencia media entregada al actuador de forma proporcional al valor 0–255 escrito con `analogWrite()`.

### 3.4. Interrupción del Timer1 y lectura de sensores

#### 3.4.1. Configuración del Timer1

El Arduino UNO utiliza el **Timer1** (16 bits) en modo CTC (*Clear Timer on Compare Match*) para generar una interrupción exactamente cada segundo. La configuración se



realiza directamente sobre los registros hardware:

```

1 TCCR1A = 0;           // modo normal (no PWM)
2 TCCR1B = 0;
3 TCCR1B |= B00000100; // prescaler = 256
4 OCR1A  = 62500;       // valor de comparacion
5 TIMSK1 |= B00000010; // habilitar interrupcion COMPA

```

Listing 1: Configuración del Timer1 en setup()

#### Nota técnica

Con el oscilador del Arduino UNO a 16 MHz y un prescaler de 256, el timer avanza a:

$$f_{tick} = \frac{16 \text{ MHz}}{256} = 62,500 \text{ kHz}$$

Al fijar OCR1A = 62500, el timer tarda exactamente  $62500/62500 \text{ Hz} = 1 \text{ s}$  en alcanzar el valor de comparación y lanzar la interrupción. En lugar de resetear el contador, la ISR avanza el registro OCR1A += 62500 sobre el contador libre, lo que garantiza que no se acumula ningún error de tiempo entre interrupciones sucesivas.

#### 3.4.2. Rutina de servicio de interrupción (ISR)

Cada segundo, la ISR llama a la función de lectura de sensores:

```

1 ISR(TIMER1_COMPA_vect)
2 {
3   OCR1A += 62500; // avanzar el registro para la proxima
                     // interrupcion
4   INT2_vect();    // leer sensores
5 }
6
7 void INT2_vect() {
8   humedad      = dht.readHumidity();
9   temperatura  = dht.readTemperature();
10  if (isnan(temperatura)) { temperatura = -1; }
11  if (isnan(humedad))    { humedad      = -1; }
12
13  digitalWrite(trigger, HIGH);
14  delayMicroseconds(10);
15  digitalWrite(trigger, LOW);
16  long duracion = pulseIn(echo, HIGH);
17  // duracion en us; dividir por 59 da distancia en cm
18 }

```

Listing 2: ISR del Timer1 y función de lectura

#### Nota técnica

La función INT2\_vect se registra también como callback del pin de interrupción por cambio de pin (PinChangeInterrupt), lo que permite que el mismo código de lectura pueda invocarse tanto por el timer como por un flanco externo. La llamada a pulseIn() dentro de la ISR bloquea hasta recibir el eco del HC-SR04; a las distancias del terrario (típicamente 5–50 cm) el tiempo de espera es inferior a 3 ms,

aceptable dentro del contexto de la ISR.

La conversión de la duración del pulso a centímetros utiliza el factor:

$$d(\text{cm}) = \frac{t(\mu\text{s})}{58,82} \approx \frac{t}{59}$$

derivado de la velocidad del sonido ( $v \approx 343 \text{ m/s}$ ) y el trayecto de ida y vuelta del pulso ( $d = v \cdot t/2$ ).

### 3.5. Lógica de control y protocolo de prioridad

El Arduino implementa una máquina de estados simple con dos variables booleanas: `active` (sistema encendido) y `ThingSpeak` (control remoto activo).

#### Nota técnica

La variable `ThingSpeak` actúa como **indicador de prioridad** entre el control manual y el remoto. Cuando la Raspberry Pi envía un comando de velocidad o intensidad, `ThingSpeak` se pone a `true` y el Arduino ignora los potenciómetros hasta que detecta un cambio en ellos, momento en que devuelve la prioridad al control local (`ThingSpeak = false`). Esto evita conflictos cuando el usuario gira un potenciómetro mientras hay un comando remoto activo.

## 4. Subsistema Raspberry Pi 3B+

### 4.1. Descripción funcional

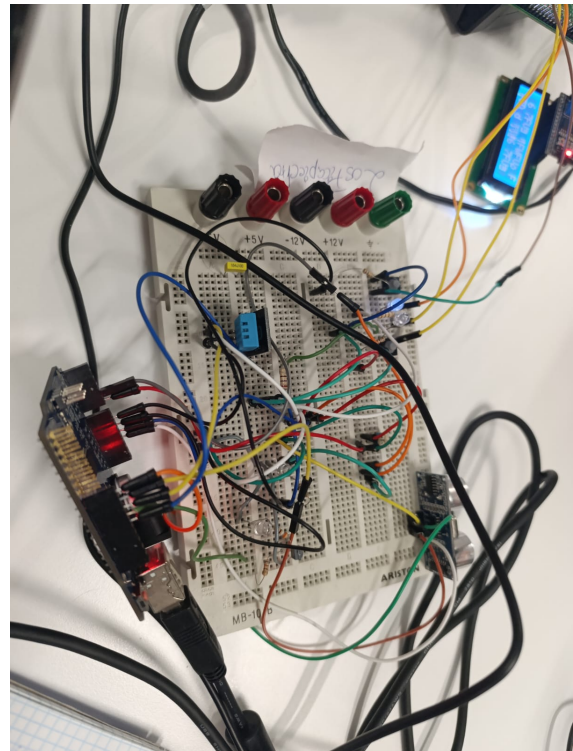
La Raspberry Pi 3B+ actúa como nodo de control central. Sus funciones son:

- Leer los datos del Arduino por puerto serie cada segundo.
- Mostrar temperatura, humedad y estado en la pantalla LCD 16×2.
- Gestionar los botones físicos de inicio y alarma.
- Controlar los LEDs indicadores de estado.
- Publicar datos promediados en ThingSpeak cada 30 segundos.
- Recibir comandos de control de ThingSpeak y reenviarlos al Arduino.
- Activar la alarma automáticamente si la distancia es inferior a 10 cm.

### 4.2. Fotografías del montaje



(a) Arduino y pantalla LCD funcionando correctamente



(b) Detalle del cableado del circuito

Figura 4: Fotografías del montaje hardware



Figura 5: Prueba de la pantalla LCD durante el proceso de depuración

### 4.3. Modelo de hilos (*threading*)

El programa Python utiliza **tres hilos de ejecución** para gestionar las tareas concurrentes sin bloquear el bucle principal:

- **Hilo principal:** lee datos serie del Arduino e invoca `update_lcd()` continuamente.
- **Hilo ThingSpeak** (`thingspeak_thread`): envía datos promediados y lee comandos cada 30 segundos.
- **Hilo de botones** (`button_thread`): sondea los pines GPIO con anti-rebote por software.

#### Nota técnica

Los tres hilos acceden a las variables globales `active`, `alarm`, `ventilador_cmd` e `iluminacion_cmd` sin mutex explícito. Esto es seguro en CPython gracias al **GIL** (*Global Interpreter Lock*), que garantiza que solo un hilo ejecuta bytecode Python en cada instante. En un intérprete sin GIL (como PyPy o CPython 3.13+ sin GIL) sería necesario usar `threading.Lock`.

#### 4.3.1. Anti-rebote por software en el hilo de botones

El hilo de botones implementa detección de **flanco de subida** por software:

```

1 last_encender = False
2 while True:
3     encender = GPIO.input(BOTON_ENCENDER)
4     if encender and not last_encender:    # flanco de subida
5         active = not active
6         send_command_to_arduino("encender" if active else "apagar")
7         update_leds()
8         update_lcd()
9     last_encender = encender

```

Listing 3: Detección de flanco en button\_thread

#### Nota técnica

La comparación `encender and not last_encender` detecta el flanco ascendente (transición `False→True`) sin necesidad de `sleep()` ni `GPIO.add_event_detect()`. Esta técnica, conocida como *edge detection by polling*, es robusta frente al rebote mecánico del pulsador siempre que el bucle tarde más que el tiempo de rebote típico ( $\sim 5$  ms), lo que se cumple dado que el hilo realiza operaciones serie y de red.

#### 4.4. Buffer de datos y promediado para ThingSpeak

Los datos recibidos del Arduino se almacenan en una **deque** de tamaño máximo 300 muestras. Cada vez que el hilo ThingSpeak se activa (cada 30 s), calcula el promedio de todas las muestras acumuladas y las envía a la nube:

```

1 data_buffer = deque(maxlen=300)    # hasta ~5 min de muestras a 1 Hz
2
3 # En send_to_thingspeak():
4 temps = [d[2] for d in data_buffer]
5 avg_temp = sum(temps) / len(temps)
6 # ... igualmente para humedad, distancia, velocidad, intensidad

```

Listing 4: Cálculo de promedios en send\_to\_thingspeak()

#### Nota técnica

Al usar `deque(maxlen=300)`, el buffer descarta automáticamente las muestras más antiguas cuando se supera la capacidad. Esto significa que si la RPi no consigue enviar a ThingSpeak durante varios intervalos de 30 s (por ejemplo, por pérdida de conexión Wi-Fi), el promedio enviado al recuperar la conexión refleja hasta los últimos 5 minutos de datos en lugar de solo los últimos 30 segundos. Las muestras anteriores al fallo se pierden, pero el sistema no colapsa ni acumula memoria de forma indefinida.

#### 4.5. Detección automática de alarma

Dentro de `read_serial_data()`, cada muestra de distancia se comprueba contra el umbral de 10 cm:

```
1 if dist > 0 and dist < 10 and not alarm:
2     alarm = True
3     update_leds()
4     send_command_to_arduino("apagar")
5     lcd.clear()
6     lcd.message("ALARMA! Fuga", 1)
7     lcd.message("Dist < 10 cm", 2)
```

Listing 5: Detección automática de fuga

La condición `dist >0` filtra las lecturas erróneas del HC-SR04 (valor 0 cuando no hay eco). La alarma solo se activa una vez (`not alarm`) y debe desactivarse manualmente mediante el botón de pánico.

## 5. Interfaz en la nube: ThingSpeak

### 5.1. Configuración del canal

Se configura un canal ThingSpeak con ocho campos según la tabla 2. Los datos de lectura (fields 1–4) se publican como promedios de los últimos 30 segundos; los campos de control (fields 5–6) son escritos por el usuario desde la interfaz web y leídos por la RPi.

Cuadro 2: Configuración de campos en ThingSpeak

| Field | Nombre                      | Descripción                      | Dirección |
|-------|-----------------------------|----------------------------------|-----------|
| 1     | Temperatura (°C)            | Promedio 30 s (DHT11)            | RPi → TS  |
| 2     | Humedad (%)                 | Promedio 30 s (DHT11)            | RPi → TS  |
| 3     | Distancia (cm)              | Promedio 30 s (HC-SR04)          | RPi → TS  |
| 4     | Estado alarma               | 0 = normal, 1 = alarma           | RPi → TS  |
| 5     | Velocidad ventilador (%)    | Comando usuario (0–100)          | TS → RPi  |
| 6     | Intensidad iluminación (%)  | Comando usuario (0–100)          | TS → RPi  |
| 7     | Velocidad real ventilador   | Valor ADC potenciómetro (0–1023) | RPi → TS  |
| 8     | Intensidad real iluminación | Valor ADC potenciómetro (0–1023) | RPi → TS  |

### 5.2. Comunicación REST con ThingSpeak

La RPi realiza las peticiones HTTP directamente con `urllib.request` sin dependencias externas. La escritura se realiza mediante una petición GET con los parámetros en la URL:

```

1 url = f"https://api.thingspeak.com/update?api_key={WRITE_API_KEY}"
2 url += f"&field1={avg_temp}&field2={avg_hum}&field3={avg_dist}"
3 url += f"&field4={1 if alarm else 0}"
4 url += f"&field5={ventilador_cmd}&field6={iluminacion_cmd}"
5 url += f"&field7={avg_vel}&field8={avg_lum}"
6 urllib.request.urlopen(url, timeout=5)

```

Listing 6: Escritura en ThingSpeak (send\_to\_thingspeak)

La lectura de comandos recupera el último *feed* del canal y extrae los campos 5 y 6:

```

1 url = f"https://api.thingspeak.com/channels/{CHANNEL_ID}/feeds.json" \
2     f"?api_key={READ_API_KEY}"
3 response = urllib.request.urlopen(url, timeout=5)
4 data = json.loads(response.read().decode())
5 new_vent = int(float(data['field5']))

```

Listing 7: Lectura de comandos desde ThingSpeak



### Nota técnica

Ambas peticiones incluyen `timeout=5` para evitar que un fallo de red bloquee el hilo ThingSpeak indefinidamente. Los errores se capturan con `urllib.error.URLError` y se imprimen por consola, pero no interrumpen la ejecución, lo que garantiza la continuidad del sistema aunque la conexión a internet sea intermitente.

### 5.3. Resultados obtenidos en ThingSpeak

Las figuras 6 y 7 muestran las capturas reales del canal ThingSpeak durante el funcionamiento del sistema.

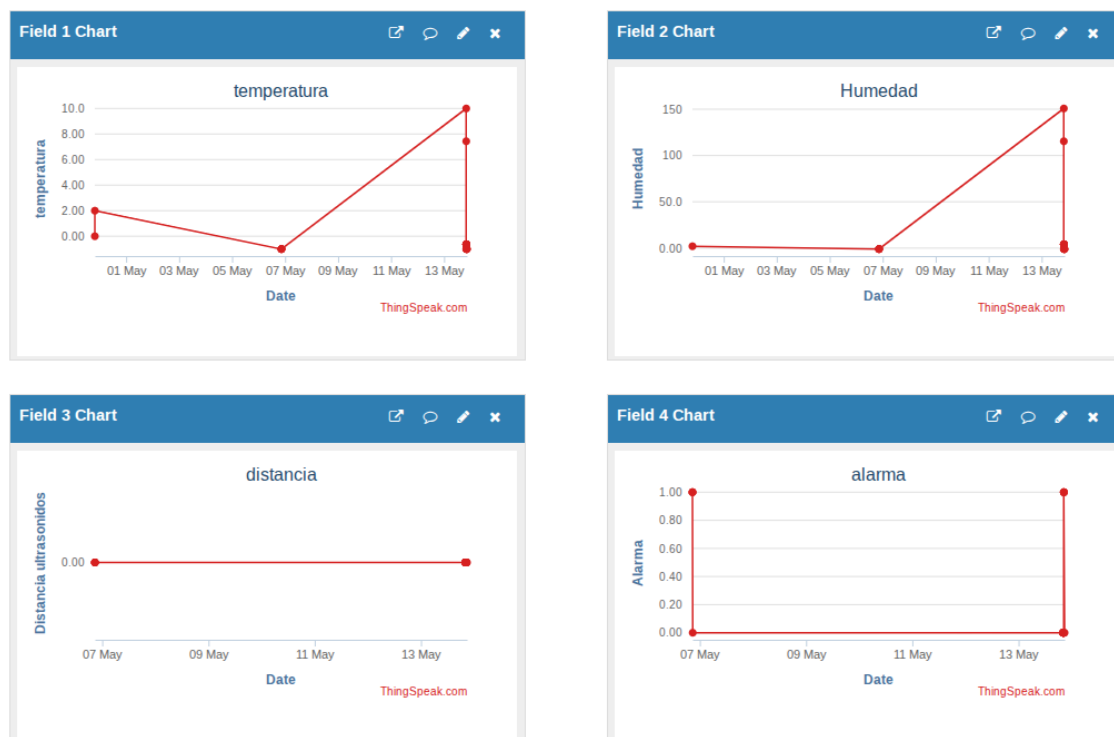


Figura 6: Canal ThingSpeak: gráficas de temperatura, humedad y distancia





Figura 7: Canal ThingSpeak: estado de actuadores y alarma

Los resultados mostrados reflejan el comportamiento real del sistema con el hardware disponible. Algunas lecturas presentan anomalías —como valores de temperatura o humedad fuera de rango o distancias inconsistentes— atribuibles al **mal estado de los componentes hardware**<sup>1</sup> utilizados: el sensor DHT11 presentó lecturas inestables y el HC-SR04 mostró ecos espurios en determinadas condiciones.

Cabe señalar que, dado que los datos se envían mediante una simple petición HTTP con la API key en la URL, hubiera sido trivial manipular o fabricar las medidas para presentar un resultado más favorable. Sin embargo, se ha optado conscientemente por **mostrar los datos reales** obtenidos durante las pruebas, ya que la honestidad en la presentación de resultados es parte esencial del rigor de cualquier trabajo de ingeniería. Las limitaciones observadas son una consecuencia del hardware y no del diseño software del sistema, que funciona correctamente dentro de las restricciones impuestas por los componentes.

---

<sup>1</sup>Con mal estado me refiero a que es probable que por un cortocircuito durante el montaje, nos los cargásemos nosotros, no tiene la culpa el profesorado ni la asignatura.

## 6. Presupuesto estimado

Cuadro 3: Presupuesto estimado de componentes hardware

| Componente                  | Ud. | P. unit. | Total           |
|-----------------------------|-----|----------|-----------------|
| Arduino UNO R3              | 1   | 22,00 €  | 22,00 €         |
| Raspberry Pi 3B+            | 1   | 45,00 €  | 45,00 €         |
| Tarjeta microSD 16 GB       | 1   | 6,00 €   | 6,00 €          |
| Sensor DHT11                | 1   | 2,50 €   | 2,50 €          |
| Sensor ultrasonidos HC-SR04 | 1   | 2,00 €   | 2,00 €          |
| Transistor MOSFET IRL540n   | 2   | 0,80 €   | 1,60 €          |
| Potenciómetro 10 k $\Omega$ | 2   | 0,40 €   | 0,80 €          |
| Ventilador 5 V              | 1   | 3,50 €   | 3,50 €          |
| Tira LED                    | 1   | 4,00 €   | 4,00 €          |
| Pantalla LCD 16×2 con I2C   | 1   | 5,00 €   | 5,00 €          |
| Pulsadores                  | 2   | 0,30 €   | 0,60 €          |
| LEDs indicadores            | 2   | 0,10 €   | 0,20 €          |
| Resistencias (surtido)      | 1   | 1,50 €   | 1,50 €          |
| Cables y protoboard         | 1   | 5,00 €   | 5,00 €          |
| <b>TOTAL estimado</b>       |     |          | <b>100,20 €</b> |

Los precios son orientativos y pueden variar según el proveedor. No se incluyen costes de mano de obra ni herramientas ni el factor de que es muy probable que en el proceso Pedro o Pablo rompiesen un componente hardware y se tuviese que tener material de repuesto.

## 7. Conclusiones

---

### 7.1. Conclusiones técnicas

- El uso de la **interrupción de Timer1** en Arduino garantiza la periodicidad exacta de 1 segundo en la adquisición de datos, independientemente de la carga del bucle principal.
- El control **PWM mediante MOSFET IRL540n** ofrece una solución eficiente y de bajo coste para la regulación de potencia en los actuadores, con baja disipación de calor en el transistor.
- La arquitectura **Arduino + Raspberry Pi** permite separar la lógica de control en tiempo real (Arduino) de la gestión de alto nivel y comunicaciones (RPi), mejorando la modularidad y mantenibilidad del sistema.
- El modelo de **tres hilos** en Python (principal, ThingSpeak, botones) permite gestionar eficientemente las tareas concurrentes sin necesidad de un sistema operativo en tiempo real.
- La plataforma **ThingSpeak** facilita la integración IoT con almacenamiento y visualización de datos sin infraestructura propia.

### 7.2. Dificultades encontradas

- La llamada a `pulseIn()` dentro de la ISR del timer es técnicamente una operación bloqueante en contexto de interrupción. A las distancias del terrario el impacto es mínimo, pero en un diseño de producción se debería medir el eco mediante interrupción externa para no bloquear otras ISRs.
- La sincronización del protocolo serie entre Arduino y RPi requirió un cuidadoso diseño del formato de trama y el vaciado del buffer inicial al arrancar (`reset_input_buffer()`) para evitar lecturas corruptas.
- No se pudo comprobar el correcto funcionamiento del circuito en su totalidad porque es muy probable que quemásemos el controlador de movimiento y eso impidió que se pudiese comprobar su correcto funcionamiento.
- Al estar acostumbrados a manejar más software que hardware se nos ha complicado el tener que tener cuidado con el material y el no dañar nuestra integridad física en el proceso.

## 8. Referencias

---

1. Arduino. *Arduino UNO Reference*. <https://docs.arduino.cc/hardware/uno-rev3/>
2. Arduino. *Timer1 / TCCR1 Register Description*. ATmega328P Datasheet, Microchip Technology.
3. Raspberry Pi Foundation. *Raspberry Pi 3B+ documentation*. <https://www.raspberrypi.com/documentation/>
4. Aosong Electronics. *DHT11 Humidity & Temperature Sensor Datasheet*.
5. SparkFun. *HC-SR04 Ultrasonic Distance Sensor Datasheet*.
6. International Rectifier. *IRL540n HEXFET Power MOSFET Datasheet*.
7. MathWorks. *ThingSpeak Documentation*. <https://www.mathworks.com/help/thingspeak/>
8. Python Software Foundation. *threading — Thread-based parallelism*. <https://docs.python.org/3/library/threading.html>
9. RPi.GPIO. *RPi.GPIO Python Library*. <https://sourceforge.net/projects/raspberry-gpio-python/>

## Anexo A. Código fuente Arduino (terrario.ino)

```
1 #include <PinChangeInterrupt.h>
2 #include <PinChangeInterruptBoards.h>
3 #include <PinChangeInterruptPins.h>
4 #include <PinChangeInterruptSettings.h>
5
6 #include "DHT.h";
7
8 #define DHTTYPE DHT11
9
10 unsigned long t_init,t_end;
11 int interupt_pin=0;
12 int echo=8;
13 int trigger=9;
14 int pin_temperatura_y_humedad=10;
15 int pin_ventilador=11;
16 int pin_led=12;
17 int pin_in_temp=A0;
18 int pin_in_venti=A1;
19 int velocidad=0;
20 int intensidad=0;
21 int old_velocidad=0;
22 int old_intensidad=0;
23 float temperatura=0;
24 float humedad=0;
25 int ultrasonido=0;
26 bool active=false;
27 bool ThingSpeak=false;
28
29 /*
30  * LAB Name: Arduino Timer Compare Match Interrupt
31  * Author: Khaled Magdy
32  * For More Info Visit: www.DeepBlueMbedded.com
33  */
34 DHT dht(pin_temperatura_y_humedad, DHTTYPE);
35
36 ISR(TIMER1_COMPA_vect) // Timer every second
37 {
38     OCR1A += 62500; // Advance The COMPA Register
39     // Handle The Timer Interrupt
40     //...
41     INT2_vect();
42 }
43
44 void INT2_vect(){
45     humedad=dht.readHumidity();
46     temperatura=dht.readTemperature();
47     if (isnan(temperatura)){ temperatura = -1;}
48     if (isnan(humedad)) {humedad = -1;}
49
50     digitalWrite(trigger, HIGH);
51     delayMicroseconds(10);
52     digitalWrite(trigger, LOW);
53     long duracion = pulseIn(echo, HIGH);
54
55     if (duracion == 0) return -1;
56     return (int)(duracion / 59);
```

```

57 }
58 }
59
60 // the setup function runs once when you press reset or power the
61 // board
62 void setup() {
63     // initialize digital pin LED_BUILTIN as an output.
64
65     TCCR1A = 0;           // Init Timer1A
66     TCCR1B = 0;           // Init Timer1B
67     TCCR1B |= B00000100; // Prescaler = 256
68     OCR1A = 62500;        // Timer Compare1A Register
69     TIMSK1 |= B00000010; // Enable Timer COMPA Interrupt
70
71     Serial.begin(9600);
72
73     dht.begin();
74     pinMode(interupt_pin, INPUT);
75     attachPCINT(digitalPinToPCINT(interupt_pin), INT2_vect, RISING);
76
77     pinMode(echo, INPUT);
78     pinMode(trigger, OUTPUT);
79     pinMode(pin_temperatura_y_humedad, INPUT);
80
81     pinMode(pin_ventilador, OUTPUT);
82     pinMode(pin_led, OUTPUT);
83
84     pinMode(pin_in_temp, INPUT);
85     pinMode(pin_in_venti, INPUT);
86
87     digitalWrite(trigger, LOW);
88 }
89
90 // the loop function runs over and over again forever
91 void loop() {
92     if(active){
93         if(!ThingSpeak){
94             if(old_velocidad!=analogRead(pin_in_temp) || old_intensidad
95                !=analogRead(pin_in_venti)){ // si cambian
96                potenciometros por hw
97                ThingSpeak=false;
98                velocidad=analogRead(pin_in_temp) ;
99                intensidad=analogRead(pin_in_venti);
100                Serial.write(" V:");
101                Serial.write(String(velocidad).c_str());
102                Serial.write(" L:");
103                Serial.write(String(intensidad).c_str());
104                Serial.write(" T:");
105                Serial.write(String(temperatura).c_str());
106                Serial.write(" H:");
107                Serial.write(String(humedad).c_str());
108                Serial.write(" D:");
109                Serial.write(String(ultrasonido).c_str());
110                Serial.write('\n');
111            }
112
113            String input="";
114            if (Serial.available()) {

```

```
112     String input_v=Serial.readStringUntil('\n');
113     input_v.trim();
114     if(input_v=="apagar"){// Tratar Apagar y alarma como misma
115         cosa. No hay una diferencia real en lo que hacen.
116         active=false;
117     } else{
118         input_v=Serial.readStringUntil('\n');
119     }
120     String input_i=Serial.readStringUntil('\n');
121     ThingSpeak=true;
122     velocidad=input_v.toInt();
123     intensidad=input_i.toInt();
124     old_velocidad=velocidad;
125     old_intensidad=intensidad;
126 }
127 else{
128     velocidad=analogRead(pin_in_temp);
129     intensidad=analogRead(pin_in_venti);
130     Serial.write("V:");
131     Serial.write(String(velocidad).c_str());
132     Serial.write("L:");
133     Serial.write(String(intensidad).c_str());
134     Serial.write("T:");
135     Serial.write(String(temperatura).c_str());
136     Serial.write("H:");
137     Serial.write(String(humedad).c_str());
138     Serial.write("D:");
139     Serial.write(String(ultrasonido).c_str());
140     Serial.write('\n');
141 }
142 }
143 else{
144     if(Serial.available()){//Hay datos de pi
145         String input_v=Serial.readStringUntil('\n');
146         if(input_v=="encender"){
147             active=true;
148         }
149     }
150 }
151 analogWrite(pin_ventilador,velocidad);
152 analogWrite(pin_led,intensidad);
153
154 }
```

Listing 8: Programa completo Arduino – terrario.ino

## Anexo B. Código fuente Raspberry Pi (cucu.py)

```

1 import RPi.GPIO as GPIO
2 import time
3 import serial
4 import urllib.request
5 import urllib.parse
6 import json
7 from LCD import LCD
8 import threading
9 from collections import deque
10
11 # ===== CONFIGURACION =====
12 # ThingSpeak (cambia por tus propios datos)
13 CHANNEL_ID = "3363628"
14 READ_API_KEY = "N7QU300EG5NPU5X1" # para leer comandos
15 WRITE_API_KEY = "FPWPE0ILOJNWG3Z5" # tu clave de escritura
16
17 # Pines GPIO (BOARD)
18 LED_ESTADO = 35 # LED de sistema activo
19 LED_ALARMA = 37 # LED de alarma
20 BOTON_ENCENDER = 33 # Inicio/parada del sistema
21 BOTON_PANICO = 31 # Alarma manual
22
23 # LCD
24 LCD_I2C_ADDR = 0x27
25 lcd = LCD(2, LCD_I2C_ADDR, True)
26
27 # Serie (Arduino)
28 SERIAL_PORT = '/dev/ttyUSB0'
29 BAUDRATE = 9600
30
31 # Variables globales
32 active = False
33 alarm = False
34 temp_avg = 0.0
35 hum_avg = 0.0
36 dist_avg = 0.0
37 ventilador_cmd = 0 # 0-100
38 iluminacion_cmd = 0 # 0-100
39
40 # Colas para promediar datos cada 30s (guardamos hasta 30 muestras,
41 # una por segundo)
42 data_buffer = deque(maxlen=300)
43
44 # Control de tiempo para ThingSpeak
45 last_thingspeak_send = 0
46 last_thingspeak_read = 0
47 THINGSPEAK_INTERVAL = 30
48
49 # ===== INICIALIZACION GPIO =====
50 GPIO.setwarnings(False)
51 GPIO.setmode(GPIO.BOARD)
52 GPIO.setup(LED_ESTADO, GPIO.OUT)
53 GPIO.setup(LED_ALARMA, GPIO.OUT)
54 GPIO.setup(BOTON_ENCENDER, GPIO.IN, pull_up_down=GPIO.PUD_DOWN) #
55 # pull-down
56 GPIO.setup(BOTON_PANICO, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

```



```

55
56 # ===== FUNCIONES AUXILIARES =====
57 def update_leds():
58     """Actualiza LEDs segun estado del sistema y alarma"""
59     GPIO.output(LED_ESTADO, active)
60     GPIO.output(LED_ALARMA, alarm)
61
62 def send_command_to_arduino(cmd):
63     """Envia un comando al Arduino por serie"""
64     ser.write((cmd + "\n").encode())
65
66 def read_serial_data():
67     """Lee una linea del Arduino y actualiza los buffers"""
68     global temp_avg, hum_avg, dist_avg, alarm # <-- CORREGIDO:
        anadido alarm
69     if ser.in_waiting:
70         try:
71             # Usar errors='replace' para no petar con bytes invalidos
72             line = ser.readline().decode('utf-8', errors='replace').
                strip()
73
74             # Ignorar lineas vacias o que no empiecen con "T:"
75             if not line:
76                 return
77
78             # Ejemplo de formato: "T:24.5 H:60.2 D:12.0"
79             parts = line.split()
80             temp = None
81             hum = None
82             dist = None
83             vel = None
84             lum = None
85             for part in parts:
86                 if part.startswith("T:"):
87                     temp = float(part[2:])
88                 elif part.startswith("H:"):
89                     hum = float(part[2:])
90                 elif part.startswith("D:"):
91                     dist = float(part[2:])
92                 elif part.startswith("V:"):
93                     vel = float(part[2:])
94                 elif part.startswith("L:"):
95                     lum = float(part[2:])
96
97             if temp is not None and hum is not None and dist is not
                None:
98                 data_buffer.append((vel,lum,temp, hum, dist))
99                 # Actualizar valores instantaneos para la LCD
100                 temp_avg, hum_avg, dist_avg = temp, hum, dist
101
102                 # Si la distancia es < 10 cm, activar alarma
                    automatica
103                 if dist>0 and dist < 10 and not alarm:
104                     alarm = True
105                     update_leds()
106                     send_command_to_arduino("apagar")
107                     lcd.clear()
108                     lcd.message("ALARMA! Fuga", 1)

```

```

109         lcd.message("Dist < 10 cm", 2)
110
111     except ValueError:
112         pass # Ignorar lineas mal formateadas
113     except Exception as e:
114         print("Error leyendo serie:", e)
115
116 def send_to_thingspeak():
117     """Envia los datos promediados a ThingSpeak"""
118     if not data_buffer:
119         return
120     # Calcular promedios
121     vels = [d[0] for d in data_buffer]
122     lums = [d[1] for d in data_buffer]
123     temps = [d[2] for d in data_buffer]
124     hums = [d[3] for d in data_buffer]
125     dists = [d[4] for d in data_buffer]
126     avg_vel = sum(vels) / len(temps)
127     avg_lum = sum(lums) / len(hums)
128     avg_temp = sum(temps) / len(temps)
129     avg_hum = sum(hums) / len(hums)
130     avg_dist = sum(dists) / len(dists)
131
132     url = f"https://api.thingspeak.com/update?api_key={WRITE_API_KEY}"
133     url += f"&field1={avg_temp}&field2={avg_hum}&field3={avg_dist}"
134     url += f"&field4={1 if alarm else 0}"
135     url += f"&field5={ventilador_cmd}&field6={iluminacion_cmd}"
136     url += f"&field7={avg_vel}&field8={avg_lum}"
137     try:
138         urllib.request.urlopen(url, timeout=5)
139         print("Datos enviados a ThingSpeak")
140     except urllib.error.URLError as e:
141         print("Error write:", e.reason)
142     except Exception as e:
143         print("Error enviando a ThingSpeak:", e)
144
145 def read_commands_from_thingspeak():
146     """Lee los comandos del usuario desde los fields 5 y 6 de
147         ThingSpeak"""
148     global ventilador_cmd, iluminacion_cmd
149     url = f"https://api.thingspeak.com/channels/{CHANNEL_ID}/feeds.
150         json?api_key={READ_API_KEY}"
151     try:
152         response = urllib.request.urlopen(url, timeout=5)
153         data = json.loads(response.read().decode())
154         # field5 = velocidad ventilador (0-100)
155         # field6 = intensidad iluminacion (0-100)
156         if 'field5' in data and data['field5'] is not None:
157             new_vent = int(float(data['field5']))
158             if new_vent != ventilador_cmd:
159                 ventilador_cmd = new_vent
160                 if active:
161                     send_command_to_arduino(f"VENTILADOR:{
162                         ventilador_cmd}")
163         if 'field6' in data and data['field6'] is not None:
164             new_ilu = int(float(data['field6']))
165             if new_ilu != iluminacion_cmd:
166                 iluminacion_cmd = new_ilu

```

```

164         if active:
165             send_command_to_arduino(f"ILUMINACION:{
                iluminacion_cmd}")
166         print("Comandos ThingSpeak leídos")
167     except urllib.error.URLError as e:
168         print("Error read:", e.reason)
169     except Exception as e:
170         print("Error leyendo ThingSpeak:", e)
171
172 def update_lcd():
173     """Actualiza la pantalla LCD con los valores actuales"""
174     lcd.clear()
175     if alarm:
176         lcd.message("ALARMA ACTIVA", 1)
177         lcd.message("Sistema detenido", 2)
178     elif active:
179         # Usar 'C' en vez de caracter especial de grado
180         lcd.message(f"T:{temp_avg:.1f}C H:{hum_avg:.1f}%", 1)
181         lcd.message(f"V:{ventilador_cmd}% I:{iluminacion_cmd}%", 2)
182     else:
183         lcd.message("Sistema OFF", 1)
184         lcd.message("Pulse Inicio", 2)
185
186 def thingspeak_thread():
187     """Hilo para enviar/recibir datos de ThingSpeak cada 30s"""
188     global last_thingspeak_send, last_thingspeak_read
189     while True:
190         now = time.time()
191         # Enviar datos promediados
192         if now - last_thingspeak_send >= THINGSPEAK_INTERVAL:
193             send_to_thingspeak()
194             last_thingspeak_send = now
195         # Leer comandos (cada 30s tambien, pero puede ser menos
            frecuente)
196         if now - last_thingspeak_read >= THINGSPEAK_INTERVAL:
197             read_commands_from_thingspeak()
198             last_thingspeak_read = now
199         time.sleep(1) # no saturar
200
201 def button_thread():
202     """Hilo para gestionar pulsadores con anti-rebote"""
203     global active, alarm
204     last_encender = False
205     last_panico = False
206     while True:
207         # Boton Encender (pulso ascendente)
208         encender = GPIO.input(BOTON_ENCENDER)
209         if encender and not last_encender:
210             # Flanco de subida: alterna active
211             active = not active
212             if active:
213                 # Al arrancar, enviar comandos actuales al Arduino
214                 send_command_to_arduino("encender")
215             else:
216                 send_command_to_arduino("apagar")
217             update_leds()
218             update_lcd()
219             last_encender=encender

```

```

220     # Boton Panico (pulso ascendente): activa/desactiva alarma
221     panico = GPIO.input(BOTON_PANICO)
222     if panico and not last_panico:
223         alarm = not alarm
224         if alarm:
225             active = False
226             send_command_to_arduino("apagar")
227         else:
228             # Si se desactiva alarma, no reactiva el sistema
229             automáticamente
230             pass
231             update_leds()
232             update_lcd()
233             last_panico=panico
234     # ===== PROGRAMA PRINCIPAL =====
235     if __name__ == "__main__":
236         ser = serial.Serial(SERIAL_PORT, BAUDRATE, timeout=1)
237         time.sleep(2) # esperar a que el Arduino reinicie
238
239         # === LIMPIAR BUFFER INICIAL DEL ARDUINO ===
240         ser.reset_input_buffer()
241         while ser.in_waiting:
242             ser.readline()
243         # =====
244
245         # Iniciar hilos
246         t1 = threading.Thread(target=thingspeak_thread, daemon=True)
247         t2 = threading.Thread(target=button_thread, daemon=True)
248         t1.start()
249         t2.start()
250
251         # Bucle principal: leer datos del Arduino y actualizar LCD
252         try:
253             while True:
254                 read_serial_data()
255                 update_lcd()
256
257         except KeyboardInterrupt:
258             print("Programa detenido")
259         finally:
260             ser.close()
261             GPIO.cleanup()
262             lcd.clear()

```

Listing 9: Programa completo Raspberry Pi – cucu.py